Studio Nx

P1-04 회원가입 API — 함수 설명 합본 (14~25)

2026-10-12

목차

1. ai-notes/backend(fastAPI_Python)/P1-04 - 회원가입 API/14_security_check_password_rules.md
2. ai-notes/backend(fastAPI_Python)/P1-04 - 회원가입 API/15_security_hash_password.md
3. ai-notes/backend(fastAPI_Python)/P1-04 - 회원가입 API/16_security_verify_password.md
4. ai-notes/backend(fastAPI_Python)/P1-04 - 회원가입 API/17_auth_service_normalize_email.md
5. ai-notes/backend(fastAPI_Python)/P1-04 - 회원가입 API/18_auth_service_release_withdrawn.md
6. ai-notes/backend(fastAPI_Python)/P1-04 - 회원가입 API/19_auth_service_check_email.md
7. ai-notes/backend(fastAPI_Python)/P1-04 - 회원가입 API/20_auth_service_check_nickname.md
8. ai-notes/backend(fastAPI_Python)/P1-04 - 회원가입 API/21_auth_service_signup.md
9. ai-notes/backend(fastAPI_Python)/P1-04 - 회원가입 API/22_auth_route_signup.md
10. ai-notes/backend(fastAPI_Python)/P1-04 - 회원가입 API/23_auth_route_check_email.md
11. ai-notes/backend(fastAPI_Python)/P1-04 - 회원가입 API/24_auth_route_check_nickname.md
12. ai-notes/backend(fastAPI_Python)/P1-04 - 회원가입 API/25_auth_schema_signup_request.md

Backend(FastAPI/Python) 14 — `check_password_rules()`

항목	내용
파일	`studionx/backend/app/core/security.py`
작업	P1-04 회원가입: 비밀번호 규칙 (PRD 01 FR-102)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	13 모델 관계 (P1-03 끝) · 15 `hash_password()`

1. 하는 일

비밀번호가 규칙 6가지 중 **무엇을 어겼는지** 이름 목록으로 돌려줍니다. 빈 목록이면 통과입니다.

`check_password_rules("abcdefgh")` → `["upper", "number", "special"]`

이름	규칙	join.html 조건 목록의 해당 줄
`length`	8 ~ 20자	8–20 characters
`upper`	대문자 1개 이상	Uppercase letter
`lower`	소문자 1개 이상	Lowercase letter
`number`	숫자 1개 이상	Number
`special`	키보드 기호 1개 이상 (`!@#$%^&*` 등)	Special character (!@#$...)
`allowed`	영문, 숫자, 키보드 기호만 허용 (한글과 공백은 불가)	(목록에는 없고, 칸 아래 오류 문구로 표시합니다)

2. 언제 쓰나

쓰는 곳	하는 일
P1-04 `signup()` (08)	가입 요청의 비밀번호를 검사합니다. 실패하면 422로 응답합니다
P1-09 `change_password()`	새 비밀번호를 같은 규칙으로 검사합니다

브라우저도 같은 규칙으로 검사합니다 (`join.js`의 `PASSWORD_RULES`, P1-07). 입력할 때마다 조건 목록에 ✓를 붙입니다. **그런데도 서버가 한 번 더 검사하는 이유**는, 브라우저 검사는 개발자 도구나 직접 만든 요청으로 건너뛸 수 있기 때문입니다.

검사에 실패하면 `signup()`이 이 목록을 그대로 담아 응답합니다.

```
422 {"error": {"code": "VALIDATION_ERROR", "message": "Check the password requirements.",
               "details": {"password": ["upper", "number", "special"]}}}
```

화면은 `details.password`에 담긴 이름으로 join.html 조건 목록의 해당 줄(`li[data-rule="upper"]` 등)을 표시할 수 있습니다.

3. 코드

import (파일 맨 위)

```
import re
import string

import jwt

from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
```

`security.py` 맨 위의 import입니다. **`re`와 `string` 두 줄을 이번에 추가했고**, 이 함수는 그 둘만 사용합니다. 둘 다 파이썬 기본 모듈이라 설치할 필요가 없습니다.

import	가져오는 것	쓰는 곳
`re`	정규식 (글자 패턴 찾기)	대문자, 소문자, 숫자, 허용 문자 검사
`string`	자주 쓰는 글자 모음	`string.punctuation`: 키보드 기호 32개

함수

```
PASSWORD_MIN, PASSWORD_MAX = 8, 20


def check_password_rules(plain: str) -> list[str]:
    failed: list[str] = []
    if not PASSWORD_MIN <= len(plain) <= PASSWORD_MAX:
        failed.append("length")
    if not re.search(r"[A-Z]", plain):
        failed.append("upper")
    if not re.search(r"[a-z]", plain):
        failed.append("lower")
    if not re.search(r"[0-9]", plain):
        failed.append("number")
    if not any(ch in string.punctuation for ch in plain):
        failed.append("special")
    if not re.fullmatch(r"[!-~]+", plain):
        failed.append("allowed")
    return failed
```

문법 풀이

import 문법	뜻
`import 모듈`	모듈 전체를 가져와서 `모듈.이름`으로 씁니다 (예: `re.search`, `string.punctuation`)

식	뜻
`PASSWORD_MIN, PASSWORD_MAX = 8, 20`	두 변수에 한 번에 값을 넣습니다 (`PASSWORD_MIN = 8`, `PASSWORD_MAX = 20`)
`8 <= len(plain) <= 20`	파이썬은 비교를 이어서 쓸 수 있습니다. `8 <= len(plain) and len(plain) <= 20`과 같습니다
`r"[A-Z]"`	정규식입니다. `[A-Z]`는 A부터 Z 중 한 글자를 뜻합니다. 앞의 `r`은 `\`를 글자 그대로 쓰겠다는 표시입니다
`re.search(패턴, 글)`	글 **어딘가에** 패턴이 있으면 결과를, 없으면 `None`을 돌려줍니다. `not None`은 `True`입니다
`re.fullmatch(패턴, 글)`	글 **전체가** 패턴과 맞아야 결과를 돌려줍니다

식	뜻
`[!-~]+`	`!`(33번)부터 `~`(126번)까지의 글자가 1개 이상이라는 뜻입니다. 영문, 숫자, 키보드 기호가 모두 이 범위 안에 있고, 한글과 공백(32번)은 범위 밖입니다
`any(ch in string.punctuation for ch in plain)`	글자를 하나씩 보면서 기호인 것이 **하나라도** 있으면 `True`입니다

입력	결과
`"Abcdef1!"`	`[]` (통과)
`"Ab1!xyz"` (7자)	`["length"]`
`"abcdefgh"`	`["upper", "number", "special"]`
`"Abcdef1!한"`	`["allowed"]`
`"Abc def1!"` (공백)	`["allowed"]`
`""`	6개 모두

4. 흐름

1. 빈 목록 `failed`를 만듭니다.
2. 규칙 6개를 **차례대로 모두** 검사하고, 어긴 규칙의 이름을 넣습니다.
3. 목록을 돌려줍니다.

5. 왜 이렇게

선택	이유
처음 실패한 규칙에서 멈추지 않고 모두 검사	화면이 어긴 조건을 한 번에 모두 표시할 수 있습니다
`True`/`False`가 아니라 이름 목록을 돌려줌	화면의 조건 줄 이름(`data-rule`)과 같아서 그대로 연결됩니다
이 함수는 오류를 던지지 않음	가입(08)과 비밀번호 변경(P1-09)이 같은 함수를 쓰고, 오류 응답은 각자 만듭니다
기호는 `string.punctuation`으로 판단	파이썬이 정해 둔 키보드 기호 32개입니다(`!"#$%&'()*+,-./:;<=>?@[\]^_{|}~`). `join.js`의 `/[!-/:-@[-{-~]/`와 같은 모음인지 시험으로 확인했습니다
최대 20자	PRD FR-102에 정해진 값입니다. bcrypt(15)는 72바이트까지만 쓰므로, 20자는 그보다 충분히 짧습니다

6. 시험 (`tests/test_password.py` 새 파일, 16개)

시험	확인한 것
통과 4개	8자, 20자, 같은 글자 반복, 밑줄 기호는 모두 `[]`입니다
실패 10개	7자, 21자, 대문자 없음, 소문자 없음, 숫자 없음, 기호 없음, 한글, 공백, 여러 규칙 동시 위반, 빈 문자열은 위 표의 목록과 같습니다
모든 기호	`string.punctuation` 32개가 각각 `special`로 인정됩니다
화면과 같은 규칙	ASCII 33~126에서 영문과 숫자를 뺀 모음이 `string.punctuation`(= `join.js` 정규식)과 같습니다

전체 시험 92개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 15 — `hash_password()`

항목	내용
파일	`studionx/backend/app/core/security.py`
작업	P1-04 회원가입: 비밀번호를 해시로 저장 (PRD 01 FR-102)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	14 `check_password_rules()` · 16 `verify_password()`

1. 하는 일

비밀번호를 **되돌릴 수 없는 문자열(해시)**로 바꿉니다. DB에는 비밀번호 대신 이 해시만 저장합니다.

`hash_password("Abcdef1!")` → `"$2b$12$Q9xK…(60자)"`

해시의 부분	예	뜻
`$2b$`		bcrypt 방식이라는 표시입니다
`12$`		비용(cost) 12입니다. 계산을 2^{12}번 반복하며, 클수록 느립니다
다음 22자	`Q9xKc3…`	소금(salt)입니다. 해시마다 새로 만든 무작위 값입니다
나머지 31자		비밀번호와 소금으로 계산한 결과입니다

2. 언제 쓰나

쓰는 곳	하는 일
P1-04 `signup()` (21)	규칙 검사(14)를 통과한 비밀번호를 해시해서 `users.password_hash`에 저장합니다
P1-09 `change_password()`	새 비밀번호를 해시해서 바꿔 저장합니다

로그인(P1-05)할 때는 이 해시를 되돌리지 않습니다. 입력한 비밀번호를 같은 소금으로 다시 계산해서 결과가 같은지 비교합니다 (다음 함수 16 `verify_password()`).

비밀번호를 그대로 저장하지 않는 이유

저장 방식	DB가 유출되면
비밀번호 그대로	모든 회원의 비밀번호가 바로 드러납니다. 같은 비밀번호를 쓰는 다른 사이트까지 위험해집니다
단순 해시 (SHA-256 등)	계산이 빨라서, 흔한 비밀번호를 초당 수십억 개씩 대입해 찾아낼 수 있습니다
bcrypt (소금 + 느린 계산)	한 번 계산하는 데 약 0.2초가 걸립니다. 또 회원마다 소금이 달라서 미리 만들어 둔 표로 찾을 수 없습니다

3. 코드

import (파일 맨 위)

```
import re
import string

import bcrypt
import jwt

from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
```

`security.py` 맨 위의 import입니다. **`import bcrypt`를 이번에 추가했고**, 이 함수는 그것만 사용합니다. `bcrypt` 패키지는 `requirements.txt`에 이미 있습니다 (설치된 버전 5.0.0).

함수

```
def hash_password(plain: str) -> str:
    return bcrypt.hashpw(plain.encode("utf-8"), bcrypt.gensalt()).decode("ascii")
```

문법 풀이

import 문법	뜻
`import 모듈`	모듈 전체를 가져와서 `모듈.이름`으로 씁니다 (예: `bcrypt.hashpw`)

한 줄짜리 코드를 안쪽부터 풀면 다음과 같습니다.

순서	식	값
①	`plain.encode("utf-8")`	`"Abcdef1!"`를 `b"Abcdef1!"`로 바꿉니다 (문자열 → 바이트. bcrypt는 바이트만 받습니다)
②	`bcrypt.gensalt()`	새 소금 `b"$2b$12$Q9xKc3…"`을 만듭니다 (기본 비용 12)
③	`bcrypt.hashpw(①, ②)`	해시 `b"$2b$12$Q9xKc3…(60바이트)"`를 계산합니다 (바이트)
④	`.decode("ascii")`	바이트를 문자열로 바꿉니다. DB의 `VARCHAR` 칸에 넣기 위해서입니다

식	뜻
`b"…"`	바이트 값입니다. 문자열(`str`)과 다른 타입입니다
`"ascii"`로 되돌림	bcrypt 해시는 영문, 숫자, `$./`만 쓰므로 ASCII로 충분합니다

4. 흐름

1. 비밀번호를 바이트로 바꿉니다.
2. 새 소금을 만듭니다.
3. 비밀번호와 소금으로 해시를 계산합니다 (약 0.2초).
4. 문자열로 바꿔서 돌려줍니다.

5. 왜 이렇게

선택	이유
bcrypt 사용	PRD FR-102가 정한 방식입니다. 오랫동안 검증된 비밀번호 전용 해시입니다
소금을 매번 새로 만듦	같은 비밀번호라도 회원마다 해시가 달라집니다. 한 사람의 비밀번호를 알아내도, 같은 비밀번호를 쓰는 다른 회원은 드러나지 않습니다

선택	이유
소금을 따로 저장하지 않음	해시 안(`$2b$12$` 뒤 22자)에 들어 있으므로, 칸 하나(`password_hash`)로 충분합니다
비용 12 (기본값)	이 컴퓨터에서 한 번에 약 0.19초가 걸립니다. 가입이나 로그인 한 번에는 느끼지 못하지만, 대입 공격에는 매우 느립니다
72바이트 제한을 따로 처리하지 않음	bcrypt 5는 72바이트가 넘으면 오류를 냅니다. 비밀번호는 14번에서 최대 20자이고 영문, 숫자, 기호(1글자 = 1바이트)만 쓰도록 검사한 뒤 들어오므로, 72바이트를 넘을 수 없습니다

6. 시험 (`tests/test_password.py`에 5개를 추가해서 모두 21개)

시험	확인한 것
모양	문자열이고, `$2b$12$`로 시작하며, 60자입니다 (`VARCHAR(255)`에 충분합니다)
매번 다름	같은 비밀번호를 두 번 해시하면 결과가 다릅니다 (소금 때문입니다)
원래 비밀번호만 맞음	`bcrypt.checkpw`로 비교하면 `"Abcdef1!"`은 맞고 `"Abcdef1?"`는 틀립니다
비밀번호가 보이지 않음	해시 안에 원래 글자가 들어 있지 않습니다
가장 긴 비밀번호 (20자)	오류 없이 해시되고, 비교도 맞습니다

비교 시험은 다음 함수(16 `verify_password`) 대신 `bcrypt.checkpw`를 직접 썼습니다.

전체 시험 97개가 통과했고, `ruff` 검사도 통과했습니다. 해시 한 번에 약 190ms가 걸립니다.

Backend(FastAPI/Python) 16 — `verify_password()`

항목	내용
파일	`studionx/backend/app/core/security.py`
작업	P1-04 회원가입 (템플릿은 P1-05에 있었지만 15번과 짝이라 함께 구현했습니다, PRD 01 FR-103)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	15 `hash_password()` · 17 `normalize_email()`

1. 하는 일

입력한 비밀번호가 DB에 저장된 해시(15)와 **맞는지** `True` 또는 `False`로 알려 줍니다.

호출	결과
`verify_password("Abcdef1!", "$2b$12$Q9xK…")` (가입할 때 쓴 비밀번호)	`True`
`verify_password("Abcdef1?", "$2b$12$Q9xK…")`	`False`

해시를 원래 비밀번호로 **되돌리는 것이 아닙니다.** 해시 안에 들어 있는 소금(15 §1)을 꺼내서, 입력한 비밀번호를 같은 방법으로 다시 계산한 뒤 결과가 같은지 봅니다.

2. 언제 쓰나

쓰는 곳	화면	하는 일
P1-05 `authenticate()`	home.html 로그인 모달	이메일로 찾은 회원의 `password_hash`와 비교합니다. `False`면 401 "Email or password is incorrect."로 응답합니다
P1-09 `check_current_password()`	userInfo.html의 비밀번호 변경과 탈퇴	지금 비밀번호를 확인합니다. `False`면 400 "Password is incorrect."로 응답합니다

3. 코드

import (파일 맨 위)

```
import re
import string

import bcrypt
import jwt

from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
```

`security.py` 맨 위의 import입니다 (15번과 같고, 이번에 새로 추가한 것은 없습니다). 이 함수는 **`bcrypt`**만 사용합니다.

함수

```
def verify_password(plain: str, hashed: str) -> bool:
    try:
        return bcrypt.checkpw(plain.encode("utf-8"), hashed.encode("ascii"))
    except ValueError:
        return False
```

문법 풀이

import 문법	뜻
`import 모듈`	모듈 전체를 가져와서 `모듈.이름`으로 씁니다 (예: `bcrypt.checkpw`)

식	뜻
`bcrypt.checkpw(비밀번호, 해시)`	해시 안의 소금으로 비밀번호를 다시 계산해서 같으면 `True`입니다. 둘 다 바이트로 넘깁니다
`plain.encode("utf-8")`, `hashed.encode("ascii")`	문자열을 바이트로 바꿉니다 (15번과 반대 방향)
`try: … except ValueError: return False`	`ValueError`가 나면 오류로 멈추지 않고 `False`를 돌려줍니다

입력	`bcrypt.checkpw`	이 함수
맞는 비밀번호	`True`	`True`
틀린 비밀번호	`False`	`False`
72바이트가 넘는 비밀번호 (`"A" * 100`)	**`ValueError`**	`False`
망가진 해시 (`""`, `"x"`)	**`ValueError`** ("Invalid salt")	`False`

4. 흐름

1. 비밀번호와 해시를 바이트로 바꿉니다.
2. bcrypt가 해시에서 소금과 비용을 꺼내 비밀번호를 다시 계산합니다 (약 0.2초).
3. 결과가 같으면 `True`, 다르면 `False`를 돌려줍니다.
4. 계산할 수 없는 입력이면 `False`를 돌려줍니다.

5. 왜 이렇게

선택	이유
`ValueError`를 `False`로 바꿈	로그인 입력은 100자까지 받습니다(`LoginRequest`). 비밀번호 규칙(14)은 가입할 때만 검사하므로, 로그인 칸에 긴 글을 넣으면 bcrypt가 오류를 던져 **500**이 납니다. 그런 비밀번호로 가입한 회원은 없으므로 "틀림"이 맞는 답입니다
망가진 해시도 `False`	DB에 잘못된 값이 있어도 로그인이 500으로 깨지지 않고 "틀림"으로 끝납니다
`==`로 직접 비교하지 않음	`bcrypt.checkpw`는 결과와 상관없이 비교에 걸리는 시간이 같습니다. 그래서 시간 차이로 해시를 추측하는 공격을 막습니다
템플릿(P1-05)보다 먼저 구현	15번과 짝이라서, 15번의 시험을 "해시 → 비교"로 함께 할 수 있습니다

P1-05에서 알아 둘 점: 이메일이 없는 회원이면 이 함수를 부르지 않아서 응답이 0.2초 빨라집니다. 그 시간 차이로 "이 이메일은 가입되어 있다"는 사실을 알아낼 수 있습니다. 그래서 `authenticate()` 템플릿에는 회원이 없을

때도 가짜 해시로 이 함수를 한 번 부르라고 적혀 있습니다.

6. 시험 (`tests/test_password.py`에 11개를 추가해서 모두 32개)

시험	확인한 것
맞는 비밀번호	`True`입니다
틀린 비밀번호 5개	한 글자 다름, 대소문자 다름, 뒤에 공백, 빈 문자열, 두 번 반복은 모두 `False`입니다
너무 긴 비밀번호	`"A" * 100`과 한글 30자(90바이트)는 오류 없이 `False`입니다
망가진 해시 4개	`""`, `"x"`, `"not-a-bcrypt-hash"`, `"$2b$12$짧음"`은 오류 없이 `False`입니다

전체 시험 108개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 17 — `normalize_email()`

항목	내용
파일	`studionx/backend/app/services/auth_service.py`
작업	P1-04 회원가입: 이메일 비교 기준 (PRD 01 FR-101)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	16 `verify_password()` · 18 `release_withdrawn()`

1. 하는 일

이메일의 **앞뒤 공백을 지우고 소문자로** 바꿉니다. 같은 사람이 다르게 입력한 이메일을 하나의 값으로 맞추기 위해서입니다.

입력	결과
`"User@Example.COM"`	`"user@example.com"`
`" user@example.com "`	`"user@example.com"`
`"first.last+tag@Mail.Example.com"`	`"first.last+tag@mail.example.com"` (점과 `+`는 그대로 둡니다)

2. 언제 쓰나

이메일을 **DB와 비교하거나 저장하기 전에는 항상** 이 함수를 거칩니다.

쓰는 곳	하는 일
19 `check_email()`	join.html 이메일 칸의 실시간 중복 확인
21 `signup()`	가입할 때의 중복 확인과 저장
P1-05 `authenticate()`	로그인 모달에서 입력한 이메일로 회원 찾기

이 함수가 없으면 생기는 문제

순서	일어나는 일
①	A가 `kim@naver.com`으로 가입합니다
②	B가 `Kim@Naver.com`으로 가입합니다. DB의 `UNIQUE`는 글자가 다르니 통과시킵니다 (MySQL 설정에 따라 다릅니다)
③	두 계정이 실제로는 **같은 메일함**을 가리킵니다. 비밀번호 찾기 메일이 엉뚱한 사람에게 갈 수 있습니다
④	또는 A가 로그인할 때 `Kim@naver.com`이라고 입력하면 회원을 찾지 못합니다

3. 코드

import (파일 맨 위)

```
from sqlalchemy.orm import Session
```

```
from app.models import User
from app.schemas.auth import AvailabilityResponse, SignupRequest
```

`auth_service.py` 맨 위의 import입니다. **이 함수는 import한 것을 쓰지 않습니다.** `strip()`과 `lower()`는 파이썬 문자열(`str`)에 기본으로 들어 있는 기능입니다.

함수

```
def normalize_email(email: str) -> str:
    return email.strip().lower()
```

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름` 만 가져와서 그대로 씁니다

식	뜻	예
`문자열.strip()`	앞뒤의 공백, 탭, 줄바꿈을 지웁니다. **가운데는 그대로 둡니다**	`" a b "` → `"a b"`
`문자열.lower()`	모두 소문자로 바꿉니다	`"Kim@Naver.COM"` → `"kim@naver.com"`
`.strip().lower()`	앞의 결과에 이어서 다음 기능을 부릅니다 (왼쪽부터 차례대로 실행됩니다)	

4. 흐름

1. 앞뒤 공백을 지웁니다.
2. 소문자로 바꿉니다.

5. 왜 이렇게

선택	이유
함수 하나로 모음	중복 확인, 가입, 로그인이 **같은 기준**으로 비교해야 합니다. 한 곳이라도 빠지면 §2의 문제가 생깁니다
`@` 앞부분까지 소문자로 바꿈	규격상 `@` 앞은 대소문자를 구분할 수 있지만, 네이버나 Gmail 같은 실제 메일 서비스는 구분하지 않습니다. 사람이 대소문자를 섞어 입력해도 같은 계정이 되는 편이 안전합니다
점과 `+`는 건드리지 않음	Gmail은 `a.b@`와 `ab@`를 같게 보지만, 다른 서비스는 그렇지 않습니다. 서비스마다 규칙이 달라서 손대지 않습니다
가운데 공백은 지우지 않음	`"a b@x.com"`은 틀린 형식입니다. 고쳐 주지 않고, 입력 검사(`EmailStr`, 25)가 422로 돌려보냅니다
화면이 보낸 값은 그대로 받고 여기서 맞춤	브라우저가 무엇을 보내든, 서버가 마지막으로 같은 기준을 지킵니다

6. 시험 (`tests/test_auth_service.py` 새 파일, 7개)

시험	확인한 것
입력 5가지	그대로, 대문자, 앞뒤 공백, 탭과 줄바꿈, 점과 `+` 유지가 위 표의 결과와 같습니다

시험	확인한 것
같은 사람의 다른 입력	`" Kim@Naver.com"`과 `"kim@naver.com "`이 같은 값이 됩니다
가운데 공백	지우지 않습니다 (`"a b@x.com"`)

전체 시험 115개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 18 — `release_withdrawn()`

항목	내용
파일	`studionx/backend/app/services/auth_service.py`
작업	P1-04 회원가입: 탈퇴 후 재가입 (PRD 01 FR-108)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	17 `normalize_email()` · 19 `check_email()`

1. 하는 일

탈퇴한 회원의 이메일과 닉네임을 **다른 값으로 바꿔서**, 같은 이메일과 닉네임으로 새로 가입할 수 있게 합니다.

칸	바꾸기 전 (탈퇴한 85번 회원)	바꾼 뒤
`email`	`kim@naver.com`	`deleted+85@withdrawn.invalid`
`nickname`	`beatmaker`	`deleted_user_0000000085`
`status`, `deleted_at`, 프로젝트		**그대로 둡니다**

이 함수가 필요한 이유

순서	일어나는 일
①	85번 회원이 탈퇴하면 행을 지우지 않고 `status=DELETED`로 바꿉니다 (P1-09, 소프트 삭제)
②	그 행은 `kim@naver.com`과 `beatmaker`를 그대로 가지고 있습니다
③	4일 뒤 같은 사람이 `kim@naver.com`으로 다시 가입하려고 하면, `email`이 `UNIQUE`라서 DB가 거절합니다
④	그래서 ③ 직전에 이 함수가 85번 행의 값을 비켜 둡니다. 그러면 새 행이 들어갈 수 있습니다

2. 언제 쓰나

"그 값으로 다시 가입하려는 사람이 나타났을 때" 부릅니다. 탈퇴한 지 3일이 지났는지는 **부르는 쪽이 확인합니다**.

쓰는 곳	화면	상황
19 `check_email()`, 20 `check_nickname()`	join.html 중복 확인	그 값을 가진 행이 탈퇴한 지 3일이 지났다면 비켜 두고 "쓸 수 있음"으로 응답합니다
21 `signup()`	join.html Sign Up 버튼	가입 직전에 같은 확인을 합니다

탈퇴 후 경과	결과 (FR-108, 2026-10-03 결정)
3일 안	비켜 두지 않고 "You can sign up again after ..."로 응답합니다
3일 지남	**이 함수**가 비켜 두어서 새로 가입할 수 있습니다

3. 코드

import (파일 맨 위)

```
from sqlalchemy.orm import Session

from app.models import User
from app.schemas.auth import AvailabilityResponse, SignupRequest
```

`auth_service.py` 맨 위의 import입니다. 이 함수는 **`Session`과 `User`**를 타입 표시에 씁니다.

함수

```
def release_withdrawn(db: Session, user: User) -> None:
    user.email = f"deleted+{user.user_id}@withdrawn.invalid"
    user.nickname = f"deleted_user_{user.user_id:010d}"
    db.flush()
```

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름`만 가져와서 그대로 씁니다

식	뜻	85번 회원일 때
`f"deleted+{user.user_id}@…"`	f-문자열입니다. `{ }` 안의 값을 글자 사이에 끼워 넣습니다	`deleted+85@withdrawn.invalid`
`{user.user_id:010d}`	정수(`d`)를 **10자리**로 쓰고, 빈자리는 `0`으로 채웁니다	`0000000085`
`db.flush()`	바꾼 값을 지금 DB에 보냅니다 (`UPDATE`). 아직 확정(`commit`)하지는 않습니다	
`-> None`	돌려주는 값이 없습니다	

4. 흐름

1. 이메일을 `deleted+<번호>@withdrawn.invalid`로 바꿉니다.
2. 닉네임을 `deleted_user_<번호 10자리>`로 바꿉니다.
3. `flush`로 DB에 먼저 반영합니다. 그래야 같은 저장 안에서 이어서 넣는 새 행이 `UNIQUE`에 걸리지 않습니다.

5. 왜 이렇게

선택	이유
행을 지우지 않고 값만 바꿈	탈퇴 회원이 남긴 프로젝트(`owner_id`)와 문의 기록이 그 행을 가리킵니다. FK 때문에 지울 수도 없습니다
회원 번호를 넣음	탈퇴 회원마다 값이 달라지므로, 비켜 둔 값끼리도 `UNIQUE`에 걸리지 않습니다
이메일 도메인을 `.invalid`로	인터넷 규격(RFC 2606)상 **실제로 존재할 수 없는** 도메인입니다. 이 주소로 메일이 갈 일이 없습니다

선택	이유
닉네임을 템플릿(`deleted_85`)과 다르게 **23자**로 만듦	`deleted_85`는 회원이 직접 가입할 수 있는 닉네임(2~20자)입니다. 누군가 먼저 그 이름을 쓰고 있다면 비켜 둘 때 `UNIQUE` 충돌이 나서 500이 됩니다. 가입 닉네임은 최대 20자이므로, 21자 이상인 이름은 **절대 겹치지 않습니다** (칸은 50자까지 받습니다)
`commit`이 아니라 `flush`	가입과 같은 저장으로 묶습니다. 가입이 실패하면 비켜 둔 것도 함께 취소됩니다
3일 확인은 부르는 쪽에서	이 함수는 "비켜 두기" 한 가지만 합니다. 언제 비켜 둘지는 19~21번이 정합니다

탈퇴 회원 추적 방지 (2026-10-07 결정)

비켜 둔 값에는 **회원 번호만** 들어 있습니다. 원래 이메일과 닉네임은 덮어써져 남지 않으므로, 이 값으로 그 사람이 누구였는지 알아낼 수는 없습니다. 회원 번호 자체도 바깥에 드러나지 않도록 다음 대책을 지킵니다.

대책	적용 시점과 위치	막는 것
① 다른 사람에게는 **'Deleted user'**로만 보내고, 그 사람의 `user_id`도 응답에 넣지 않습니다	Phase 6 멤버 목록, 채팅, 댓글 (`user_service.withdraw` TODO에 기록)	`deleted_user_0000000085`가 화면에 보여서 회원 번호가 드러나는 것
② 3일 뒤 남은 개인 정보도 지웁니다: `password_hash`는 `"!"`로 바꾸고, `last_login_at`과 `profile_image_path`는 비웁니다	보관 기간 정리 `retention.py` (P1-09, TODO에 기록)	탈퇴한 행에 개인 정보가 계속 남는 것
③ 회원 번호로 다른 회원을 찾는 공개 API를 만들지 않습니다	지금 API를 확인했습니다. 회원 번호는 가입 응답과 내 정보(본인)에만 나갑니다	번호를 알더라도 그 번호로 아무것도 조회할 수 없게 합니다

DB를 직접 보는 관리자(SQLAdmin, P1-21)는 회원 번호와 탈퇴한 행을 볼 수 있습니다. 관리 목적상 필요하며, 관리자 화면에는 ADMIN 계정만 들어갈 수 있습니다.

6. 시험 (`tests/test_auth_service.py`에 4개를 추가해서 모두 11개, 실제 MySQL로 실행하고 rollback)

시험	확인한 것
비켜 둔 뒤 같은 값으로 가입	이메일과 닉네임이 바뀌고, 같은 `gone@example.com`과 `gone_nick`으로 새 회원이 들어갑니다
비켜 두지 않은 경우	같은 이메일로 넣으면 DB가 거절합니다 (`IntegrityError`). 이 함수가 필요한 이유입니다
행과 프로젝트는 그대로	`status=DELETED`와 `deleted_at`이 그대로이고, 남긴 프로젝트의 `owner_id`도 그대로입니다
닉네임 충돌 없음	비켜 둔 닉네임이 20자보다 길고 50자 이하입니다. 회원이 `deleted_85` 같은 닉네임을 쓰고 있어도 충돌하지 않습니다

전체 시험 119개가 통과했고, `ruff` 검사도 통과했습니다. 시험이 끝난 뒤 남은 행은 0개입니다.

Backend(FastAPI/Python) 19 — `check_email()` • `availability_of()`

항목	내용
파일	`studionx/backend/app/services/auth_service.py`
작업	P1-04 회원가입: 이메일 중복 확인과 탈퇴 후 3일 규칙 (PRD 01 FR-101, FR-108)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	18 `release_withdrawn()` · 20 `check_nickname()`

1. 하는 일

이 이메일로 **새로 가입할 수 있는지** 알려 줍니다.

DB에서 그 이메일을 가진 행	결과	join.html 이메일 칸 아래 문구
없음	`{"available": true}`	✓ Available
활동 중(`ACTIVE`)이거나 정지된(`SUSPENDED`) 회원	`{"available": false, "reason": "TAKEN"}`	This email is already in use.
탈퇴한 지 **3일이 안 됨**	`{"available": false, "reason": "REJOIN_TOO_SOON", "available_at": "2026-10-09T03:00:00Z"}`	You can sign up again after Oct 9.
탈퇴한 지 3일이 지남	`{"available": true}` (가입할 때 18번으로 비켜 둡니다)	✓ Available

함수	역할
`check_email()`	이메일로 행을 찾습니다
`availability_of()`	찾은 행을 보고 위 표대로 판단합니다. 20번 `check_nickname()`도 같은 판단을 씁니다

2. 언제 쓰나

부르는 곳	`release`	하는 일
`GET /auth/check-email` (24): join.html 이메일 칸의 **Check** 버튼	`False`	확인만 하고, DB는 바꾸지 않습니다
21 `signup()`: **Sign Up** 버튼	`True`	3일이 지난 탈퇴 행이면 18번으로 비켜 둔 뒤 가입합니다

3일 규칙이 있는 이유 (FR-108, 2026-10-03 결정): 탈퇴 직후 다른 사람이 같은 이메일이나 닉네임으로 바로 가입하면, 탈퇴한 사람인 척할 수 있습니다. 그래서 3일 동안은 그 값을 쓰지 못하게 막습니다.

3. 코드

import (파일 맨 위)

```
from datetime import UTC, datetime, timedelta
```

```
from sqlalchemy import select
from sqlalchemy.orm import Session

from app.models import User
from app.models.user import UserStatus
from app.schemas.auth import AvailabilityResponse, SignupRequest
```

`auth_service.py` 맨 위의 import입니다. **`datetime` 줄과 `select`, `UserStatus`를 이번에 추가했습니다.** 이 두 함수는 `SignupRequest`를 뺀 나머지를 모두 사용합니다.

import	가져오는 것	쓰는 곳
`UTC`, `datetime`, `timedelta` (파이썬 기본)	시간대, 시각, 기간	지금 시각, 탈퇴 시각 + 3일
`select`	SQL `SELECT`를 파이썬으로 씁니다	이메일로 행 찾기
`UserStatus`	08번의 회원 상태 목록	`DELETED`인지 확인
`AvailabilityResponse`	결과의 모양 (`schemas/auth.py`)	돌려주는 값

함수

```
REJOIN_WAIT_DAYS = 3


def check_email(db: Session, email: str, release: bool = False) -> AvailabilityResponse:
    user = db.scalar(select(User).where(User.email == normalize_email(email)))
    return availability_of(db, user, release)


def availability_of(db: Session, user: User | None, release: bool) -> AvailabilityResponse:
    if user is None:
        return AvailabilityResponse(available=True)
    if user.status != UserStatus.DELETED:
        return AvailabilityResponse(available=False, reason="TAKEN")
    if user.deleted_at is not None:
        available_at = user.deleted_at + timedelta(days=REJOIN_WAIT_DAYS)
        now = datetime.now(UTC).replace(tzinfo=None)  # DB 의 시각은 UTC · 시간대 정보 없음
        if now < available_at:
            return AvailabilityResponse(
                available=False,
                reason="REJOIN_TOO_SOON",
                available_at=available_at.replace(tzinfo=UTC),
            )
    if release:
        release_withdrawn(db, user)
    return AvailabilityResponse(available=True)
```

문법 풀이

import 문법	뜻
`from 모듈 import A, B, C`	한 모듈에서 여러 이름을 한 줄로 가져옵니다

식	뜻
`release: bool = False`	기본값이 있는 인자입니다. 넘기지 않으면 `False`(중복 확인 API)이고, 가입할 때는 `release=True`로 부릅니다

식	뜻
`select(User).where(User.email == …)`	`SELECT * FROM users WHERE email = …` 입니다
`db.scalar(…)`	결과 **한 행**을 `User`로 가져오고, 없으면 `None`입니다 (`email`은 `UNIQUE`라서 많아야 1행입니다)
`User | None`	회원 또는 `None`입니다
`timedelta(days=3)`	3일이라는 기간입니다. 시각에 더하면 3일 뒤의 시각이 됩니다
`datetime.now(UTC).replace(tzinfo=None)`	지금 UTC 시각에서 시간대 표시를 뗍니다. DB에서 읽은 `deleted_at`에도 표시가 없어서, 둘을 비교하려면 모양을 맞춰야 합니다
`available_at.replace(tzinfo=UTC)`	화면에 보낼 때는 UTC 표시를 붙여서 JSON에 `…Z`로 나가게 합니다. 화면은 이것을 사용자 시간대로 바꿔 보여 줍니다

4. 흐름

1. 이메일을 17번으로 정리한 뒤, 그 이메일을 가진 행을 찾습니다.
2. 없으면 쓸 수 있다고 응답합니다.
3. 탈퇴한 회원이 아니면(활동 중이거나 정지) `TAKEN`으로 응답합니다.
4. 탈퇴했고 3일이 지나지 않았으면 `REJOIN_TOO_SOON`과 언제부터 쓸 수 있는지를 응답합니다.
5. 3일이 지났으면 (`release`가 참이면 18번으로 비켜 두고) 쓸 수 있다고 응답합니다.

5. 왜 이렇게

선택	이유
`release` 인자를 추가함 (템플릿에 없음)	중복 확인은 `GET` 요청입니다. `GET`은 "보기만" 하는 요청이라 DB를 바꾸면 안 됩니다 (새로 고침이나 미리 불러오기로 여러 번 불릴 수 있습니다). 실제로 비켜 두는 일은 가입할 때만 합니다
판단 부분을 `availability_of()`로 분리	닉네임(20)도 규칙이 같습니다. 한 곳만 고치면 둘 다 바뀝니다
정지(`SUSPENDED`)된 회원도 `TAKEN`	정지는 탈퇴가 아닙니다. 계정이 살아 있으니 이메일도 그 사람의 것입니다
`deleted_at`이 비어 있으면 3일이 지난 것으로 봄	정상이라면 탈퇴할 때 항상 기록됩니다(P1-09). 값이 없는 이상한 행 때문에 그 이메일이 영원히 막히지 않게 합니다
`available_at`을 UTC로 보냄	서버는 시각을 모두 UTC로 다룹니다(`docs/DB.md`). 한국 사용자와 미국 사용자 모두 자기 시간대로 볼 수 있습니다
이유를 `TAKEN`과 `REJOIN_TOO_SOON`으로 나눔	화면이 서로 다른 문구를 보여 줄 수 있습니다. "이미 사용 중"과 "며칠 뒤 가능"은 사용자가 할 일이 다르기 때문입니다

참고: 이 확인은 "그 이메일이 가입되어 있는지"를 누구에게나 알려 줍니다(이메일 존재 확인). 실시간 중복 확인 기능(FR-101)을 만들면 피할 수 없는 성질이라, 대부분의 사이트도 같은 방식을 씁니다. 마구 묻는 것을 막는 요청 횟수 제한은, P1-12의 제한 방식을 함께 쓰는 방안을 그때 검토합니다.

6. 시험 (`tests/test_auth_service.py`에 8개를 추가해서 모두 19개, 실제 MySQL로 실행하고 rollback)

시험	확인한 것
없는 이메일	`available=True`
활동 회원, 정지 회원 (2개)	`TAKEN`
대소문자와 공백을 섞어서 물음	`" Taken@Example.COM "`도 `TAKEN`입니다 (17번으로 정리한 뒤 비교)
탈퇴한 지 2일	`REJOIN_TOO_SOON`이고, `available_at`은 탈퇴 시각 + 3일이며, UTC 표시가 붙어 있습니다
탈퇴한 지 3일 1시간, 확인만 함	`available=True`이고, 이메일은 **그대로**입니다
탈퇴한 지 4일, `release=True`	`available=True`이고, 이메일이 `deleted+…`로 바뀌며, 같은 이메일로 바로 가입됩니다
응답 모양	`{"available": true, "reason": null, "available_at": null}`

전체 시험 127개가 통과했고, `ruff` 검사도 통과했습니다. 시험이 끝난 뒤 남은 행은 0개입니다.

Backend(FastAPI/Python) 20 — `check_nickname()`

항목	내용
파일	`studionx/backend/app/services/auth_service.py`
작업	P1-04 회원가입: 닉네임 중복 확인 (PRD 01 FR-101, FR-108)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	19 `check_email()` · 21 `signup()`

1. 하는 일

이 닉네임을 **새로 쓸 수 있는지** 알려 줍니다. 판단 규칙은 19번과 같습니다 (`availability_of()`를 함께 씁니다).

DB에서 그 닉네임을 가진 행	결과
없음	`{"available": true}`
활동 회원, 정지 회원	`{"available": false, "reason": "TAKEN"}`
탈퇴한 지 3일이 안 됨	`{"available": false, "reason": "REJOIN_TOO_SOON", "available_at": "…Z"}`
탈퇴한 지 3일이 지남	`{"available": true}` (가입할 때 18번으로 비켜 둡니다)

대소문자가 달라도 같은 닉네임으로 봅니다

이미 있는 닉네임	확인하는 닉네임	결과
`beatmaker`	`beatmaker`	`TAKEN`
`beatmaker`	`BeatMaker`, `BEATMAKER`	`TAKEN`
`beatmaker`	`" beatmaker "` (앞뒤 공백)	`TAKEN`
`café`	`cafe`	`TAKEN` (악센트도 구분하지 않습니다)

2. 언제 쓰나

부르는 곳	`release`	화면
`GET /auth/check-nickname` (24)	`False`	join.html 닉네임 칸의 **Check** 버튼
21 `signup()`	`True`	**Sign Up** 버튼
P1-08 `update_me()`	`False` 또는 `True`	userInfo.html의 닉네임 바꾸기 (그때 정합니다)

3. 코드

import (파일 맨 위)

```
from datetime import UTC, datetime, timedelta

from sqlalchemy import select
from sqlalchemy.orm import Session
```

```
from app.models import User
from app.models.user import UserStatus
from app.schemas.auth import AvailabilityResponse, SignupRequest
```

`auth_service.py` 맨 위의 import입니다 (19번과 같고, 이번에 새로 추가한 것은 없습니다). 이 함수는 **`select`, `Session`, `User`, `AvailabilityResponse`**를 쓰고, 판단은 19번의 `availability_of()`에 맡깁니다.

함수

```
def check_nickname(db: Session, nickname: str, release: bool = False) -> AvailabilityResponse:
    user = db.scalar(select(User).where(User.nickname == nickname.strip()))
    return availability_of(db, user, release)
```

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름`만 가져와서 그대로 씁니다

식	뜻
`nickname.strip()`	앞뒤 공백만 지웁니다. 이메일과 달리 **대소문자는 바꾸지 않습니다**
`User.nickname == …`	SQL `WHERE nickname = …`입니다. 실제 비교는 **MySQL이 하며**, 아래의 정렬 규칙을 따릅니다

MySQL의 비교 규칙 (정렬 규칙 `utf8mb4_0900_ai_ci`)

`users.nickname`과 `users.email` 칸은 이 규칙을 씁니다. 아래는 이 DB에서 직접 확인한 결과입니다.

비교	결과	뜻
`'Beat' = 'beat'`	같음	`ci`는 case-insensitive, 즉 대소문자를 구분하지 않는다는 뜻입니다
`'café' = 'cafe'`	같음	`ai`는 accent-insensitive, 즉 악센트를 구분하지 않는다는 뜻입니다
`'ａ' = 'a'`	같음	전각 글자도 같게 봅니다
`'비트' = '비트 '`	**다름**	뒤 공백을 그대로 비교합니다 (`NO PAD`). 그래서 `strip()`이 필요합니다

4. 흐름

1. 앞뒤 공백을 지웁니다.
2. 그 닉네임을 가진 행을 DB에서 찾습니다 (대소문자와 악센트는 무시됩니다).
3. 19번의 `availability_of()`로 판단합니다.

5. 왜 이렇게

선택	이유
닉네임은 소문자로 바꾸지 않음	닉네임은 화면에 보이는 이름입니다. `BeatMaker`로 가입한 사람의 표기를 그대로 지켜 줍니다
그래도 `BeatMaker`와 `beatmaker`는 같은 이름으로 봄	DB의 `UNIQUE`가 이미 그렇게 비교합니다. 다르게 확인하면 "✓ Available"이라고 해 놓고 가입할 때 DB가 거절하는 일이 생깁니다. 또 `beatmaker`를 흉내 낸 `BeatMaker` 계정(사칭)을 막는 효과도 있습니다

선택	이유
비교를 파이썬이 아니라 DB에 맡김	확인 결과와 가입할 때의 DB 판단이 항상 같아집니다
앞뒤 공백은 직접 지움	MySQL은 뒤 공백을 다르게 봅니다(`NO PAD`). `"beatmaker "`로 따로 가입되면, 화면에서는 똑같아 보이는 이름이 둘 생깁니다
판단은 19번과 같은 함수로	탈퇴 3일 규칙과 정지 회원 처리가 이메일과 똑같아야 합니다

6. 시험 (`tests/test_auth_service.py`에 8개를 추가해서 모두 27개, 실제 MySQL로 실행하고 rollback)

시험	확인한 것
없는 닉네임	`available=True`
같은 닉네임	`TAKEN`
공백과 대소문자 (3개)	`" beatmaker "`, `BeatMaker`, `BEATMAKER`가 모두 `TAKEN`입니다
DB도 같은 규칙인지	`beatmaker`가 있을 때 `BeatMaker`로 넣으면 DB가 거절합니다 (`IntegrityError`). 확인 결과와 일치합니다
탈퇴한 지 1일	`REJOIN_TOO_SOON`
탈퇴한 지 5일	확인만 하면 그대로 두고, `release=True`면 `deleted_user_…`로 비켜 둔 뒤 같은 닉네임으로 가입됩니다

전체 시험 135개가 통과했고, `ruff` 검사도 통과했습니다. 시험이 끝난 뒤 남은 행은 0개입니다.

Backend(FastAPI/Python) 21 — `signup()` · `unavailable_error()`

항목	내용
파일	`studionx/backend/app/services/auth_service.py`
작업	P1-04 회원가입: 가입 처리 (PRD 01 FR-101, FR-102, FR-108, 처리방침 동의)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	20 `check_nickname()` · 22 `POST /auth/signup`

1. 하는 일

join.html의 **Sign Up** 버튼으로 들어온 가입 요청을 검사하고, 통과하면 `users`에 새 회원을 저장합니다. 14번부터 20번까지를 모두 엮는 함수입니다.

함수	역할
`signup()`	검사하고 저장한 뒤, 저장된 `User`를 돌려줍니다
`unavailable_error()`	19번과 20번의 "쓸 수 없음" 결과를 409 오류로 바꿉니다

경우	결과
모두 통과	새 `User`를 돌려줍니다 (회원 번호와 가입 시각은 DB가 채웁니다)
비밀번호 규칙 위반	422 `VALIDATION_ERROR` "Check the password requirements."와 `details.password: ["upper", …]`
처리방침에 동의하지 않음	422 "Please agree to the Privacy Policy."
예전 처리방침 화면에서 보냄	422 "Please reload the page."
이메일이 사용 중	409 `EMAIL_TAKEN` "This email is already in use."
닉네임이 사용 중	409 `NICKNAME_TAKEN` "This nickname is already in use."
탈퇴한 지 3일이 안 됨	409 `REJOIN_TOO_SOON` "You can sign up with this email again after Oct 9, 2026 (UTC)."

2. 언제 쓰나

`POST /api/v1/auth/signup`(22)이 이 함수를 부릅니다. 화면 흐름은 다음과 같습니다.

순서	화면 또는 서버	일어나는 일
①	join.html	이메일과 닉네임의 **Check** 버튼(19, 20)을 눌러 ✓ Available을 확인합니다
②	join.html	비밀번호 조건 목록이 모두 ✓가 되고, "만 16세 이상 + 처리방침 동의"에 체크합니다
③	join.js (P1-07)	**Sign Up**을 누르면 `POST /auth/signup` `{email, nickname, password, privacy_agreed, privacy_version}`을 보냅니다
④	**이 함수**	같은 검사를 서버에서 한 번 더 하고 저장합니다
⑤	join.js	201을 받으면 홈으로 이동해서 로그인 모달을 엽니다 (P1-07)

①과 ②를 화면에서 했더라도 ④에서 **다시 검사합니다**. 화면 검사는 건너뛸 수 있고, ①과 ③ 사이에 다른 사람이 그 이메일로 먼저 가입할 수도 있기 때문입니다.

3. 코드

import (파일 맨 위)

```
from datetime import UTC, datetime, timedelta

from sqlalchemy import select
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.core.security import check_password_rules, hash_password
from app.models import User
from app.models.user import UserStatus
from app.schemas.auth import PRIVACY_VERSION, AvailabilityResponse, SignupRequest
```

`auth_service.py` 맨 위의 import입니다. **`IntegrityError`, `AppError`, `ErrorCode`, `check_password_rules`, `hash_password`, `PRIVACY_VERSION`을 이번에 추가했습니다.**

import	가져오는 것	쓰는 곳
`IntegrityError`	DB가 `UNIQUE` 같은 규칙을 어긴 요청을 거절할 때의 오류	동시 가입 처리
`AppError`, `ErrorCode`	01번의 오류	422, 409
`check_password_rules`, `hash_password`	14번, 15번	비밀번호 검사와 해시
`PRIVACY_VERSION`	지금 처리방침 버전 `"2026-10-05"` (`schemas/auth.py`)	동의한 판 확인
`SignupRequest`	가입 요청의 모양	`data`

함수

```
def signup(db: Session, data: SignupRequest) -> User:
    failed = check_password_rules(data.password)
    if failed:
        raise AppError(ErrorCode.VALIDATION_ERROR, 422, "Check the password requirements.",
                       details={"password": failed})
    if not data.privacy_agreed:
        raise AppError(ErrorCode.VALIDATION_ERROR, 422, "Please agree to the Privacy Policy.",
                       details={"privacy_agreed": "required"})
    if data.privacy_version != PRIVACY_VERSION:
        raise AppError(ErrorCode.VALIDATION_ERROR, 422, "Please reload the page.",
                       details={"privacy_version": "outdated"})

    email = normalize_email(data.email)
    nickname = data.nickname.strip()
    email_result = check_email(db, email, release=True)
    if not email_result.available:
        raise unavailable_error("email", email_result)
    nickname_result = check_nickname(db, nickname, release=True)
    if not nickname_result.available:
        raise unavailable_error("nickname", nickname_result)
```

```
    user = User(
        email=email,
        nickname=nickname,
        password_hash=hash_password(data.password),
        privacy_agreed_at=datetime.now(UTC).replace(tzinfo=None),
        privacy_version=PRIVACY_VERSION,
    )
    db.add(user)
    try:
        db.commit()
    except IntegrityError as exc:
        # 동시에 같은 값으로 가입한 사람이 먼저 저장된 경우 (위 검사는 둘 다 통과)
        db.rollback()
        if "uq_users_nickname" in str(exc.orig):
            field = "nickname"
        else:
            field = "email"
        raise unavailable_error(field, AvailabilityResponse(available=False, reason="TAKEN")) from None
    db.refresh(user)
    return user


def unavailable_error(field: str, result: AvailabilityResponse) -> AppError:
    if result.reason == "REJOIN_TOO_SOON":
        when = result.available_at
        day = f"{when:%b} {when.day}, {when.year}"  # 예: Oct 9, 2026
        return AppError(ErrorCode.REJOIN_TOO_SOON, 409,
                        f"You can sign up with this {field} again after {day} (UTC).",
                        details={field: "REJOIN_TOO_SOON", "available_at": when.isoformat()})
    if field == "email":
        return AppError(ErrorCode.EMAIL_TAKEN, 409, "This email is already in use.")
    return AppError(ErrorCode.NICKNAME_TAKEN, 409, "This nickname is already in use.")
```

(실제 파일에서는 `ruff`가 `AppError(...)`의 인자를 한 줄에 하나씩 나눠 적었습니다. 내용은 같습니다.)

문법 풀이

import 문법	뜻
`from sqlalchemy.exc import IntegrityError`	라이브러리 안의 `exc`(오류 모음) 모듈에서 오류 종류 하나를 가져옵니다

식	뜻
`if failed:`	목록이 비어 있으면 거짓이고, 하나라도 있으면 참입니다
`raise unavailable_error(…)`	함수가 **만들어서 돌려준** 오류를 던집니다 (`unavailable_error`는 오류를 던지지 않고 돌려주기만 합니다)
`db.add(user)` → `db.commit()`	저장을 대기시킨 뒤 확정합니다. 확정할 때 DB가 `UNIQUE`를 마지막으로 검사합니다
`except IntegrityError as exc:`	DB가 거절하면 그 오류를 `exc`로 받습니다
`str(exc.orig)`	MySQL이 보낸 원래 오류 문구입니다. `Duplicate entry 'newbie' for key 'users.uq_users_nickname'`처럼 어느 제약인지가 들어 있습니다
`db.rollback()`	실패한 저장을 취소합니다. 하지 않으면 세션을 더 쓸 수 없습니다

식	뜻
`raise … from None`	DB 오류를 이어 붙이지 않고 이 프로젝트의 오류만 던집니다 (05번과 같음)
`db.refresh(user)`	DB가 채운 값(`user_id`, `created_at`, `role`, `status` …)을 다시 읽습니다
`f"{when:%b}"`	날짜를 영어 월 약자(`Oct`)로 씁니다
`when.isoformat()`	`"2026-10-09T03:00:00+00:00"` 처럼, 화면(JS)이 읽기 쉬운 표준 모양으로 씁니다

4. 흐름

1. 비밀번호 규칙(14), 처리방침 동의, 처리방침 판을 확인합니다. **DB를 보지 않는** 검사를 먼저 합니다.
2. 이메일(19)과 닉네임(20)을 확인합니다. 3일이 지난 탈퇴 행이면 비켜 둡니다(18).
3. 새 `User`를 만듭니다. 이메일은 17번으로 정리하고, 비밀번호는 15번으로 해시하며, 동의 시각은 지금(UTC)으로 적습니다.
4. 저장(`commit`)합니다. DB가 거절하면(동시 가입) 409로 응답합니다.
5. DB가 채운 값을 읽어서 돌려줍니다.

5. 왜 이렇게

선택	이유
DB를 보지 않는 검사를 먼저 함	틀린 요청은 DB에 묻기 전에 끝냅니다. 비켜 두기(18)도 다른 검사를 모두 통과한 뒤에만 일어납니다
중간에 실패하면 `commit` 하지 않음	이메일을 비켜 둔 뒤 닉네임에서 막혀도, 요청이 끝날 때 세션이 닫히면서 비켜 둔 것까지 취소됩니다
`role`, `status`, 용량은 넣지 않음	DB 기본값(`USER`, `ACTIVE`, 5GB)이 정합니다 (08번과 같은 원칙)
`privacy_agreed_at`은 서버 시각으로 기록	화면이 보낸 시각은 믿을 수 없습니다. 동의 기록은 서버가 남깁니다 (처리방침 D3, D4)
`privacy_version` 확인	처리방침이 바뀐 뒤 예전 화면에서 보낸 가입이라면, 새 판을 보고 다시 동의하게 합니다
`IntegrityError`를 409로 바꿈	①의 확인과 ③의 저장 사이에 다른 사람이 먼저 저장하면, 확인은 둘 다 통과합니다. 마지막에 DB의 `UNIQUE`가 막고, 그것을 500이 아니라 "이미 사용 중"으로 바꿔 응답합니다
날짜에 "(UTC)" 표시	서버는 사용자의 시간대를 모릅니다. 화면은 `details.available_at`으로 사용자 시간대의 날짜를 따로 보여 줄 수 있습니다
응답에 비밀번호나 해시를 넣지 않음	이 함수는 `User`를 돌려주지만, 라우터(22)는 `SignupResponse`(번호, 이메일, 닉네임)만 내보냅니다

참고: 닉네임의 2~20자 검사는 앞뒤 공백을 지우기 **전**의 길이로 합니다(스키마). 그래서 `" a "`는 3자로 통과한 뒤 `"a"`(1자)가 됩니다. 25번(입력 스키마)에서 공백을 먼저 지우고 길이를 재도록 고칩니다.

6. 시험 (`tests/test_auth_service.py`에 10개를 추가해서 모두 37개, 실제 MySQL로 실행)

시험	확인한 것
가입 성공	이메일은 소문자, 닉네임은 공백이 지워지고, 해시가 저장되며(원래 비밀번호가 아니고, 16번으로 맞음), `USER`와 `ACTIVE`, 동의 판과 시각이 기록되고, DB에 저장되어 있습니다

시험	확인한 것
비밀번호 규칙	422이고, `details.password = ["upper", "number", "special"]` 입니다
동의하지 않음, 예전 판	422와 각각의 문구
이메일 사용 중	409 `EMAIL_TAKEN`
닉네임이 대소문자만 다름	409 `NICKNAME_TAKEN`
탈퇴한 지 1일	409 `REJOIN_TOO_SOON`이고, 문구에 "Oct 9, 2026 (UTC)" 모양의 날짜와 `details.available_at`이 있습니다
탈퇴한 지 4일	같은 이메일과 닉네임으로 가입에 성공하고, 예전 행은 `deleted+…`와 `deleted_user_…`가 됩니다
동시 가입 (이메일, 닉네임 2개)	확인을 통과한 것처럼 꾸민 뒤 저장하면 DB가 막고, 500이 아니라 409로 응답합니다

시험 세션 변경 (`tests/conftest.py`): 이 함수는 `commit`을 합니다. 그래서 시험의 `db` 세션을 "바깥 트랜잭션 + 저장점(SAVEPOINT)" 방식으로 바꿨습니다. 코드 안의 `commit`은 저장점만 확정하고, 시험이 끝나면 바깥 트랜잭션을 통째로 되돌립니다. 앞서 만든 모델과 서비스 시험도 모두 이 방식으로 통과합니다.

전체 시험 145개가 통과했고, `ruff` 검사도 통과했습니다. 시험이 끝난 뒤 남은 행은 0개입니다.

Backend(FastAPI/Python) 22 — 라우터 `POST /api/v1/auth/signup`

항목	내용
파일	`studionx/backend/app/api/v1/auth.py` (함수 이름 `signup`)
작업	P1-04 회원가입: 가입 API (PRD 01 FR-101, §7)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	21 `signup()` · 23 `GET /auth/check-email`

1. 하는 일

브라우저가 보낸 가입 요청을 받아 21번 `auth_service.signup()`에 넘기고, 결과를 **응답 모양**으로 바꿔서 돌려줍니다.

```
POST /api/v1/auth/signup
{"email": "Route.User@Example.com", "nickname": "route_user", "password": "Abcdef1!",
 "privacy_agreed": true, "privacy_version": "2026-10-05"}

→ 201 Created
{"user_id": 92, "email": "route.user@example.com", "nickname": "route_user"}
```

2. 언제 쓰나

join.html의 **Sign Up** 버튼을 누르면 `join.js`(P1-07)가 이 주소로 요청을 보냅니다. 201을 받으면 화면은 `home.html#login`으로 옮겨 로그인 모달을 엽니다 (가입한 뒤 자동으로 로그인시키지는 않습니다).

라우터와 서비스의 차이

층	파일	하는 일
라우터 (이 함수)	`api/v1/auth.py`	주소, 방식(`POST`), 응답 상태(201), 응답 모양을 정합니다. HTTP를 아는 층입니다
서비스 (21)	`services/auth_service.py`	실제 검사와 저장을 합니다. HTTP를 모르기 때문에 시험이나 다른 곳에서도 그대로 부를 수 있습니다

요청 하나가 지나가는 길

순서	처리하는 쪽	일어나는 일
①	FastAPI	본문 JSON을 `SignupRequest`와 비교합니다. 칸이 빠졌거나 모양이 틀리면 **라우터가 실행되기 전에** 422로 응답합니다 (02번 처리기)
②	FastAPI	`db: Db`를 보고 `get_db()`로 DB 세션을 엽니다
③	**이 함수**	`auth_service.signup(db, data)`를 부릅니다
④	21	검사하고 저장합니다. 실패하면 `AppError`를 던지고, 01번 처리기가 409나 422로 응답합니다
⑤	**이 함수**	`User`를 `SignupResponse`(번호, 이메일, 닉네임만)로 바꿉니다

순서	처리하는 쪽	일어나는 일
⑥	FastAPI	201과 JSON으로 응답합니다. 요청이 끝나면 `get_db()`가 세션을 닫습니다

결과	상태	예
가입됨	201	`{"user_id": 92, …}`
칸이 빠지거나 모양이 틀림	422	`details: {"privacy_agreed": "Field required"}`
비밀번호 규칙 위반, 동의하지 않음	422	`details: {"password": ["upper", …]}`
이메일이나 닉네임이 사용 중, 탈퇴한 지 3일이 안 됨	409	`code: "EMAIL_TAKEN"` 등

3. 코드

import (파일 맨 위)

```
from typing import Annotated

from fastapi import APIRouter, Cookie, Query, Response, status

from app.api.deps import Db
from app.schemas.auth import (
    NICKNAME_MAX,
    AvailabilityResponse,
    LoginRequest,
    SignupRequest,
    SignupResponse,
    TokenResponse,
)
from app.schemas.common import ErrorResponse
from app.services import auth_service
```

`auth.py` 맨 위의 import입니다 (P1-05 로그인에서 쓰는 것까지 함께 있습니다). **`from app.services import auth_service`를 이번에 추가했습니다.** 이 함수는 `APIRouter`(→ `router`), `status`, `Db`, `SignupRequest`, `SignupResponse`, `ErrorResponse`, `auth_service`를 사용합니다.

import	쓰는 곳
`router = APIRouter()`	`@router.post(…)`로 이 파일의 주소들을 묶습니다. `api/v1/router.py`가 이것을 `/auth` 아래에 붙입니다
`status.HTTP_201_CREATED`	성공 상태 201
`Db`	DB 세션 (P1-03 `deps.py`)
`SignupRequest`, `SignupResponse`	요청과 응답의 모양
`ErrorResponse`	API 문서(`/docs`)에 409와 422 응답 모양을 보여 줍니다
`auth_service`	21번의 `signup()`을 `auth_service.signup`으로 부릅니다

함수

```
@router.post(
    "/signup",
```

```
    response_model=SignupResponse,
    status_code=status.HTTP_201_CREATED,
    responses={409: {"model": ErrorResponse}, 422: {"model": ErrorResponse}},
)
def signup(data: SignupRequest, db: Db) -> SignupResponse:
    user = auth_service.signup(db, data)
    return SignupResponse(user_id=user.user_id, email=user.email, nickname=user.nickname)
```

위의 `@router.post(…)` 부분은 템플릿에 있었고, 함수 몸통 두 줄을 이번에 채웠습니다.

문법 풀이

import 문법	뜻
`from app.services import auth_service`	**모듈(파일) 전체**를 이름 하나로 가져와서 `auth_service.signup`처럼 씁니다. 이 파일의 라우터 함수 이름도 `signup`이라서, 서비스 함수를 `from … import signup`으로 가져오면 이름이 겹칩니다

식	뜻
`@router.post("/signup", …)`	데코레이터입니다. 아래 함수를 `POST /signup` 요청에 연결합니다
`response_model=SignupResponse`	응답을 이 모양으로 **걸러서** 보냅니다. 다른 칸이 있어도 나가지 않습니다
`status_code=201`	성공하면 200이 아니라 201(만들어짐)로 응답합니다
`responses={409: …, 422: …}`	응답 설명을 API 문서에만 추가합니다 (동작에는 영향이 없습니다)
`data: SignupRequest`	요청 본문 JSON을 이 모양으로 읽어서 넣어 줍니다 (①)
`db: Db`	DB 세션을 넣어 줍니다 (②)

4. 흐름

1. FastAPI가 본문을 검사하고 DB 세션을 엽니다.
2. 21번 `signup()`으로 가입시킵니다.
3. 번호, 이메일, 닉네임만 담아서 201로 응답합니다.

5. 왜 이렇게

선택	이유
라우터는 두 줄	검사와 저장은 서비스(21)에 둡니다. 라우터가 얇으면 HTTP 없이도 서비스를 시험할 수 있습니다
`SignupResponse`로 바꿔서 응답	`User`를 그대로 내보내면 `password_hash` 같은 칸이 섞일 위험이 있습니다. 응답 모양을 정해 두면 그 칸만 나갑니다 (PRD 01 §5)
201로 응답	새로 만들었다는 뜻의 REST 관례입니다. 화면은 201을 보고 "가입 성공"을 압니다
오류는 여기서 잡지 않음	21번이 던진 `AppError`를 01번 처리기가 받아 모양을 맞춥니다. 라우터마다 `try`를 쓰지 않아도 됩니다
가입한 뒤 토큰을 주지 않음	로그인은 P1-05가 맡는 일입니다. 가입과 로그인을 나눠 두면 각각이 단순해집니다

6. 시험 (`tests/test_auth_routes.py`에 4개를 추가해서 모두 6개, 실제 앱과 실제 MySQL로 실행하고 rollback)

시험에서는 앱의 `get_db`를 시험용 세션으로 바꿔 끼우고(`app.dependency_overrides`), 실제 주소로 요청했습니다.

시험	확인한 것
가입	201이고, 응답 칸이 정확히 `user_id`, `email`, `nickname`이며(비밀번호와 해시 없음), 이메일은 소문자이고, DB에 저장되어 있습니다
같은 이메일로 다시 가입	두 번째는 409 `EMAIL_TAKEN`입니다
비밀번호 규칙	422이고, `details.password = ["upper", "number", "special"]`입니다
칸이 빠짐	422이고, `details = {"privacy_agreed": "Field required"}`입니다 (02번 처리기)

전체 시험 149개가 통과했고, `ruff` 검사도 통과했습니다. 시험이 끝난 뒤 남은 행은 0개입니다.

Backend(FastAPI/Python) 23 — 라우터 `GET /api/v1/auth/check-email`

항목	내용
파일	`studionx/backend/app/api/v1/auth.py` (함수 이름 `check_email`), `requirements.txt`
작업	P1-04 회원가입: 이메일 실시간 중복 확인 API (PRD 01 FR-101, FR-108)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	22 `POST /auth/signup` · 24 `GET /auth/check-nickname`

1. 하는 일

주소의 `?email=` 값을 받아 **형식을 검사**하고, 19번 `check_email()`의 결과를 그대로 돌려줍니다.

요청	응답
`GET /api/v1/auth/check-email?email=free@example.com`	200 `{"available": true, "reason": null, "available_at": null}`
`…?email=ROUTE.user@example.COM` (이미 가입됨)	200 `{"available": false, "reason": "TAKEN", …}`
`…?email=gone@example.com` (탈퇴한 지 1일)	200 `{"available": false, "reason": "REJOIN_TOO_SOON", "available_at": "2026-10-09T03:00:00Z"}`
`…?email=abc` (형식이 틀림)	**422** `{"error": {"code": "VALIDATION_ERROR", "details": {"email": "value is not a valid email address: An email address must have an @-sign."}}}`
… (`email`이 없음)	422

2. 언제 쓰나

join.html 이메일 칸 옆의 **Check** 버튼을 누르면 `join.js`(P1-07)가 이 주소로 묻습니다.

응답	이메일 칸 아래 문구 (P1-07)
`available: true`	✓ Available
`TAKEN`	This email is already in use.
`REJOIN_TOO_SOON`	You can sign up again after Oct 9. (`available_at`을 사용자 시간대로 바꿔 보여 줍니다)
422	Enter a valid email address.

형식이 틀리면 422로 응답하는 이유 (2026-10-07 결정): "이미 사용 중"(200, `available: false`)과 "형식이 틀림"(422)을 구분해서 다른 문구를 보여 주기 위해서입니다. 가입(22)도 같은 `EmailStr` 규칙을 쓰므로, 여기서 통과한 이메일은 가입할 때도 형식 때문에 막히지 않습니다.

3. 코드

import (파일 맨 위)

```
from typing import Annotated

from fastapi import APIRouter, Cookie, Query, Response, status
from pydantic import EmailStr

from app.api.deps import Db
from app.schemas.auth import (
    NICKNAME_MAX,
    AvailabilityResponse,
    LoginRequest,
    SignupRequest,
    SignupResponse,
    TokenResponse,
)
from app.schemas.common import ErrorResponse
from app.services import auth_service
```

`auth.py` 맨 위의 import입니다. **`from pydantic import EmailStr`를 이번에 추가했습니다.** 이 함수는 `Annotated`, `Query`, `EmailStr`, `Db`, `AvailabilityResponse`, `auth_service`를 사용합니다.

`requirements.txt`에 한 줄 추가 (새 패키지)

```
email-validator>=2.2       # pydantic EmailStr — 이메일 형식 검사 (P1-04, 2026-10-07)
```

`EmailStr`은 pydantic에 들어 있지만, 실제 검사는 이 패키지가 합니다. 설치된 버전은 2.3.0입니다. 다른 컴퓨터에서는 pull한 뒤 의존성을 다시 설치하면(CLAUDE.md) 자동으로 설치됩니다.

함수

```
@router.get("/check-email", response_model=AvailabilityResponse)
def check_email(db: Db, email: Annotated[EmailStr, Query(max_length=255)]) -> AvailabilityResponse:
    return auth_service.check_email(db, email)
```

템플릿은 `Annotated[str, …]`였는데, `str`을 `EmailStr`로 바꾸고 몸통 한 줄을 채웠습니다.

문법 풀이

import 문법	뜻
`from pydantic import EmailStr`	pydantic의 "이메일 문자열" 타입을 가져옵니다

식	뜻
`@router.get("/check-email", …)`	`GET /api/v1/auth/check-email`에 연결합니다. `GET`은 보기만 하는 요청입니다
`Annotated[EmailStr, Query(max_length=255)]`	이 인자는 **주소의 `?email=`**에서 읽고(`Query`), 255자 이하이며 이메일 형식(`EmailStr`)이어야 합니다. 틀리면 FastAPI가 라우터를 실행하기 전에 422로 응답합니다 (02번 처리기)
`auth_service.check_email(db, email)`	19번을 `release=False`(기본값)로 부릅니다. 그래서 DB를 바꾸지 않습니다
`response_model=AvailabilityResponse`	응답의 모양입니다. `available_at`은 UTC 시각이라서 `…Z`로 나갑니다

`EmailStr`이 받는 것과 막는 것 (직접 확인했습니다)

입력	결과
`kim@naver.com`	통과합니다
`" kim@naver.com "`	통과하고, 앞뒤 공백을 지워 줍니다
`Kim@Naver.COM`	통과하고 `Kim@naver.com`이 됩니다 (도메인만 소문자로 바꾸고, `@` 앞은 17번이 소문자로 바꿉니다)
`abc`, `a b@x.com`, `kim@naver`, `user@localhost`	422
`deleted+85@withdrawn.invalid`	422입니다. `.invalid`는 쓸 수 없는 도메인이므로, **18번에서 비켜 둔 이메일로는 누구도 가입할 수 없습니다**
`한글@naver.com`	통과합니다. 국제 메일 규격(SMTPUTF8)상 올바른 형식이기 때문입니다

4. 흐름

1. FastAPI가 `?email=`을 읽어 길이와 형식을 검사합니다. 틀리면 422로 응답합니다.
2. 19번으로 쓸 수 있는 이메일인지 확인합니다 (DB는 바꾸지 않습니다).
3. 결과를 200으로 돌려줍니다.

5. 왜 이렇게

선택	이유
`GET`으로 확인만 함 (`release=False`)	19번에서 정한 대로입니다. `GET`은 여러 번 불려도 DB가 바뀌면 안 됩니다
"쓸 수 없음"도 200으로 응답	요청 자체는 성공했고, 답이 "아니요"인 것입니다. 오류(4xx)는 요청 자체가 잘못되었을 때(형식이 틀림) 씁니다
형식 검사를 라우터 입구(`EmailStr`)에서 함	틀린 값으로 DB에 묻지 않습니다. 오류 모양도 02번 처리기가 다른 입력 오류와 똑같이 만듭니다
`email-validator` 패키지 추가	정규식으로 흉내 내면 `a..b@x.com`이나 국제 도메인 같은 경우를 놓칩니다. 이메일 규격을 따르는 검사를 씁니다 (2026-10-06 사용자 결정 "EmailStr")

6. 시험 (`tests/test_auth_routes.py`에 10개를 추가해서 모두 16개, 실제 앱과 실제 MySQL로 실행하고 rollback)

시험	확인한 것
없는 이메일	200, `{"available": true, "reason": null, "available_at": null}`
가입된 이메일을 대소문자 섞어서 물음	`TAKEN`
탈퇴한 지 1일	`REJOIN_TOO_SOON`이고, `available_at`이 `Z`(UTC)로 끝납니다
형식이 틀린 5개	`abc`, `a b@x.com`, `kim@naver`, `deleted+85@withdrawn.invalid`, 빈 문자열은 422 `VALIDATION_ERROR`이고, `details`에 `email`이 담깁니다
`email`이 없음	422

시험	확인한 것
탈퇴한 지 5일 된 행이 있을 때	`available: true`이지만, 그 행의 이메일은 **그대로**입니다 (GET은 확인만 합니다)

전체 시험 159개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 24 — 라우터 `GET /api/v1/auth/check-nickname`

항목	내용
파일	`studionx/backend/app/api/v1/auth.py` (함수 이름 `check_nickname`)
작업	P1-04 회원가입: 닉네임 실시간 중복 확인 API (PRD 01 FR-101, FR-108)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	23 `GET /auth/check-email` · 25 입력 스키마

1. 하는 일

주소의 `?nickname=` 값을 받아 **길이를 검사**하고, 20번 `check_nickname()`의 결과를 그대로 돌려줍니다. 23번과 같은 모양입니다.

요청	응답
`GET /api/v1/auth/check-nickname?nickname=fresh_nick`	200 `{"available": true, …}`
`…?nickname=ROUTE_USER` (`route_user`가 이미 있음)	200 `{"available": false, "reason": "TAKEN", …}`
`…?nickname=a` (1자), 21자, 빈 문자열	**422**이고, `details.nickname`에 오류가 담깁니다

2. 언제 쓰나

화면	버튼
join.html (P1-07)	닉네임 칸 옆의 **Check**
userInfo.html (P1-10)	닉네임을 바꾸기 전에 확인할 때 같은 API를 씁니다

응답에 따른 문구는 23번과 같습니다 (✓ Available / This nickname is already in use. / You can use it again after …).

3. 코드

import (파일 맨 위)

```
from typing import Annotated

from fastapi import APIRouter, Cookie, Query, Response, status
from pydantic import EmailStr

from app.api.deps import Db
from app.schemas.auth import (
    NICKNAME_MAX,
    NICKNAME_MIN,
    AvailabilityResponse,
    LoginRequest,
    SignupRequest,
    SignupResponse,
    TokenResponse,
```

```
)
from app.schemas.common import ErrorResponse
from app.services import auth_service
```

`auth.py` 맨 위의 import입니다. **`NICKNAME_MIN`을 이번에 추가했습니다.** 이 함수는 `Annotated`, `Query`, `Db`, `NICKNAME_MIN`, `NICKNAME_MAX`, `AvailabilityResponse`, `auth_service`를 사용합니다.

함수

```
@router.get("/check-nickname", response_model=AvailabilityResponse)
def check_nickname(
    db: Db,
    nickname: Annotated[str, Query(min_length=NICKNAME_MIN, max_length=NICKNAME_MAX)],
) -> AvailabilityResponse:
    return auth_service.check_nickname(db, nickname)
```

템플릿에는 `Query(max_length=NICKNAME_MAX)`만 있었습니다. **`min_length=NICKNAME_MIN`을 추가**하고 몸통 한 줄을 채웠습니다.

문법 풀이

import 문법	뜻
`from app.schemas.auth import (…, NICKNAME_MIN, …)`	괄호 안에 한 줄에 하나씩 적은 여러 이름을 가져옵니다

식	뜻
`Query(min_length=2, max_length=20)`	주소의 `?nickname=`에서 읽고, 2~20자여야 합니다. 틀리면 라우터를 실행하기 전에 422로 응답합니다 (02번 처리기)
`auth_service.check_nickname(db, nickname)`	20번을 `release=False`(기본값)로 부릅니다. 그래서 DB를 바꾸지 않습니다

4. 흐름

1. FastAPI가 `?nickname=`을 읽어 길이를 검사합니다. 틀리면 422로 응답합니다.
2. 20번으로 쓸 수 있는 닉네임인지 확인합니다 (앞뒤 공백을 지우고, 대소문자를 무시하며, DB는 바꾸지 않습니다).
3. 결과를 200으로 돌려줍니다.

5. 왜 이렇게

선택	이유
`min_length`를 추가함	템플릿대로라면 `"a"`(1자)를 물었을 때 "✓ Available"이 나오지만, 가입(22)은 2자 미만이라서 422로 막습니다. 확인과 가입의 규칙이 같아야 "쓸 수 있다더니 가입이 안 된다"는 일이 생기지 않습니다
나머지는 23번과 같음	확인만 하고(`GET`), "쓸 수 없음"도 200으로 응답하며, 형식이나 길이 오류만 422로 응답합니다

남은 차이 (25번에서 고칩니다): 지금은 앞뒤 공백을 **지우기 전**의 글자 수로 길이를 잽니다. 그래서 `" a "`는 5자로 통과한 뒤 `"a"`로 확인됩니다. 25번에서 "공백을 먼저 지우고 길이를 재는" 닉네임 타입을 만들어서, 가입 스키마와 이 라우터가 함께 쓰게 합니다.

6. 시험 (`tests/test_auth_routes.py`에 8개를 추가해서 모두 24개, 실제 앱과 실제 MySQL로 실행하고 rollback)

시험	확인한 것
없는 닉네임	200, `available: true`
가입된 `route_user`를 3가지 방식으로 물음	`route_user`, `ROUTE_USER`, `" route_user "`가 모두 `TAKEN`입니다
길이가 틀린 3개	1자, 21자, 빈 문자열은 422이고, `details`에 `nickname`이 담깁니다
`nickname`이 없음	422

전체 시험 167개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 25 — 입력 스키마 `SignupRequest` · `Nickname`

항목	내용
파일	`studionx/backend/app/schemas/auth.py`, `studionx/backend/app/api/v1/auth.py`
작업	P1-04 회원가입: 가입 입력 형식 검사 (PRD 01 FR-101, P1-04 마지막 작업)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	24 `GET /auth/check-nickname` · 26 `create_access_token()` (P1-05)

1. 하는 일

가입 요청(22)과 닉네임 확인(24)의 **입력 모양**을 정합니다. 형식이 틀리면 라우터에 닿기 전에 FastAPI가 422로 돌려보냅니다.

칸	바꾸기 전 (템플릿)	바꾼 뒤
`email`	`str` (아무 문자열)	`EmailStr`: 이메일 형식을 검사하고, 앞뒤 공백을 지우며, 254자 이하만 받습니다
`nickname`	`str` 2~20자 (공백을 포함한 길이)	`Nickname`: 앞뒤 공백을 **먼저 지운 뒤** 2~20자인지 검사합니다

입력	바꾸기 전	바꾼 뒤
`nickname: " a "`	통과(5자)하고, 저장은 `"a"`(1자)로 됩니다	**422**입니다 (공백을 빼면 1자)
`nickname: " ab "`	통과하고, 서비스가 `"ab"`로 바꿉니다	통과하고, 스키마가 `"ab"`로 바꿉니다
`email: "kim@naver"`	통과해서 DB에 이상한 이메일이 들어갑니다	**422**
`email: " kim@naver.com "`	통과하고, 서비스가 공백을 지웁니다	통과하고, 스키마가 공백을 지웁니다

2. 언제 쓰나

쓰는 곳	사용 방식
22 `POST /auth/signup`	`data: SignupRequest`로 본문 JSON을 이 모양으로 검사합니다
24 `GET /auth/check-nickname`	`nickname: Annotated[Nickname, Query()]`로 주소의 `?nickname=`을 같은 타입으로 검사합니다
23 `GET /auth/check-email`	이미 `EmailStr`을 쓰고 있습니다 (23번에서 바꿨습니다)

확인과 가입이 같은 타입을 쓰는 이유: 중복 확인에서 "✓ Available"이라고 해 놓고 가입할 때 형식 오류가 나면 안 됩니다. 같은 타입을 쓰면 규칙이 한 곳에만 있게 됩니다.

3. 코드

import (`schemas/auth.py` 맨 위)

```
from datetime import datetime
from typing import Annotated

from pydantic import BaseModel, EmailStr, Field, StringConstraints
```

`Annotated`, `EmailStr`, `StringConstraints`를 이번에 추가했습니다.

import	가져오는 것	쓰는 곳
`Annotated` (파이썬 기본)	타입에 조건을 덧붙이는 도구	`Nickname`
`EmailStr`	이메일 형식 타입 (23번에서 설치한 `email-validator`가 검사합니다)	`email`
`StringConstraints`	글자 조건 묶음 (공백 제거, 길이)	`Nickname`

함수 (`schemas/auth.py`)

```
NICKNAME_MIN, NICKNAME_MAX = 2, 20

# 닉네임 입력 타입 — 앞뒤 공백을 먼저 지우고 길이를 잼 (가입 · 중복 확인이 함께 씀, 2026-10-07)
Nickname = Annotated[
    str,
    StringConstraints(strip_whitespace=True, min_length=NICKNAME_MIN, max_length=NICKNAME_MAX),
]


class SignupRequest(BaseModel):
    email: EmailStr = Field(max_length=255)  # 형식 검사 · 앞뒤 공백 제거 (email-validator)
    nickname: Nickname
    password: str = Field(min_length=1, max_length=100)  # 정확한 8~20자 검사는 check_password_rules
    privacy_agreed: bool
    privacy_version: str = Field(max_length=20)
```

(`api/v1/auth.py`의 24번 라우터)

```
    nickname: Annotated[Nickname, Query()],     # 바꾸기 전: Annotated[str, Query(min_length=…, max_lengt
h=…)]
```

24번에서 넣었던 `min_length`와 `max_length`는 `Nickname` 안으로 들어갔기 때문에, 라우터의 `NICKNAME_MIN`과 `NICKNAME_MAX` import는 지웠습니다.

문법 풀이

import 문법	뜻
`from pydantic import BaseModel, EmailStr, Field, StringConstraints`	pydantic에서 네 이름을 한 줄로 가져옵니다

식	뜻
`Nickname = Annotated[str, StringConstraints(…)]`	**타입에 이름을 붙입니다**. "조건이 붙은 문자열"을 `Nickname` 한 단어로 여러 곳에서 씁니다
`strip_whitespace=True`	검사하기 **전에** 앞뒤 공백을 지웁니다. 지운 값이 그대로 `data.nickname`에 들어갑니다
`min_length`, `max_length`	공백을 지운 **뒤**의 길이입니다

식	뜻
`EmailStr = Field(max_length=255)`	이메일 형식과 255자 이하를 함께 검사합니다. 실제로는 이메일 규격의 상한인 254자에서 먼저 막힙니다
`Annotated[Nickname, Query()]`	같은 타입을 주소 값(`?nickname=`)에도 씁니다

4. 흐름 (요청이 들어올 때)

1. FastAPI가 JSON과 주소 값을 꺼냅니다.
2. `email`은 앞뒤 공백을 지운 뒤 형식을 검사하고, `nickname`은 앞뒤 공백을 지운 뒤 2~20자인지 검사합니다.
3. 하나라도 틀리면 02번 처리기가 422와 `details`(어느 칸이 틀렸는지)로 응답합니다.
4. 모두 맞으면 정리된 값(공백을 지운 이메일과 닉네임)이 라우터를 거쳐 서비스(21)로 갑니다.

5. 왜 이렇게

선택	이유
형식 검사는 스키마(입구)에서 함	틀린 입력이 서비스나 DB까지 가지 않습니다. 오류 모양도 다른 입력 오류와 같아집니다 (02)
비밀번호는 스키마에서 1~100자만 검사	8~20자와 조건 검사는 14번이 합니다. 조건별 실패 목록(`["upper", …]`)을 화면에 돌려주려면 서비스에서 검사해야 합니다
이메일 소문자 변환은 스키마가 아니라 17번에서	로그인(P1-05)과 중복 확인도 17번을 거칩니다. 비교 기준을 한 함수에 모아 둡니다
서비스(21)의 `strip()`과 `normalize_email()`은 그대로 둠	스키마를 거치지 않고 서비스를 부르는 곳(시험, 나중의 관리 기능)이 있어도 같은 기준이 지켜집니다
`Nickname`을 `schemas/auth.py`에 둠	P1-08 회원 정보 수정(`schemas/user.py`)에서도 닉네임을 바꿉니다. 그때 이 타입을 가져다 쓰면 됩니다

6. 시험

`tests/test_auth_schemas.py` (새 파일, 16개)

시험	확인한 것
닉네임 통과 4개	`beatmaker`, `" beatmaker "` → `beatmaker`, `" ab "` → `ab`, 20자
닉네임 실패 5개	1자, `" a "`, 공백만, 21자, 공백을 빼도 21자는 모두 `nickname` 오류입니다
이메일 공백	`" kim@naver.com "`은 `kim@naver.com`이 됩니다
이메일 실패 5개	`abc`, `a b@x.com`, `kim@naver`, `deleted+85@withdrawn.invalid`, 빈 문자열은 모두 `email` 오류입니다
이메일 길이	254자는 통과하고, 300자가 넘으면 `email` 오류입니다

`tests/test_auth_routes.py`에 3개 추가 (모두 27개)

시험	확인한 것
`GET /check-nickname?nickname= a`	422입니다 (24번에서 남아 있던 차이가 고쳐졌습니다)
`POST /signup` 닉네임 `" a "`	422이고, `details`에 `nickname`이 담깁니다
`POST /signup` 이메일 `kim@naver`	422이고, `details`에 `email`이 담깁니다

전체 시험 186개가 통과했고, `ruff` 검사도 통과했습니다.